

```

import numpy as np
import matplotlib.pyplot as plt

# -----
# Step 1: Given data
# -----
X_pos = np.array([[3,1],[3,-1],[6,1],[6,-1]])
X_neg = np.array([[1,0],[0,1],[0,-1],[-1,0]])

# Support vectors (from problem)
s1 = np.array([1,0])
s2 = np.array([3,1])
s3 = np.array([3,-1])

# Labels
y1, y2, y3 = -1, 1, 1

# -----
# Step 2: Augment vectors (add bias term 1)
# -----
s1_hat = np.append(s1, 1)
s2_hat = np.append(s2, 1)
s3_hat = np.append(s3, 1)

S = np.array([s1_hat, s2_hat, s3_hat])
Y = np.array([y1, y2, y3])

# -----
# Step 3: Solve for alpha
# -----
A = np.zeros((3,3))

for i in range(3):
    for j in range(3):
        A[j][i] = np.dot(S[i], S[j])

alpha = np.linalg.solve(A, Y)

a1, a2, a3 = alpha

# -----
# Step 4: Compute weight vector
# -----
w = a1*s1_hat + a2*s2_hat + a3*s3_hat

w1, w2, b = w

# -----
# Step 5: Print results
# -----

```

```

print("Alpha values:")
print("a1 =", a1)
print("a2 =", a2)
print("a3 =", a3)

print("\nWeight vector (w):", (w1, w2))
print("Bias (b):", b)

print("\nEquation:")
print(f"{w1}*x + {w2}*y + ({b}) = 0")

print("\nDecision Boundary: x = 2")
print("Margins: x = 1 and x = 3")

print("\nSupport Vectors:")
print(s1, s2, s3)

# -----
# Step 6: Plot graph
# -----
plt.scatter(X_pos[:,0], X_pos[:,1], label='Positive (+1)')
plt.scatter(X_neg[:,0], X_neg[:,1], label='Negative (-1)')

# Decision boundary (vertical line x=2)
plt.axvline(x=2, label='Decision Boundary')

# Margins
plt.axvline(x=1, linestyle='--')
plt.axvline(x=3, linestyle='--')

# Highlight support vectors
sv = np.array([s1, s2, s3])
plt.scatter(sv[:,0], sv[:,1], s=120,
            facecolors='none', edgecolors='black',
            label='Support Vectors')

plt.legend()
plt.grid()
plt.xlabel('X1')
plt.ylabel('X2')
plt.title('SVM Complete Solution')
plt.show()

Alpha values:
a1 = -3.5
a2 = 0.75
a3 = 0.75

Weight vector (w): (np.float64(1.0), np.float64(0.0))
Bias (b): -2.0

```

Equation:  
 $1.0 \cdot x + 0.0 \cdot y + (-2.0) = 0$

Decision Boundary:  $x = 2$   
Margins:  $x = 1$  and  $x = 3$

Support Vectors:  
[1 0] [3 1] [ 3 -1]

